Exam Questions
Deep Reinforcement Learning

Stefan Eder

June 14, 2026

How this file is organized

A practice set for the second half of the course: credit assignment, imitation learning and offline RL, inverse RL, preference-based RL / RLHF, planning, model-based RL, and RL for foundation models. **Real** old-exam questions (the Inverse RL exam and the credit-assignment quiz) are reproduced verbatim with their answer choices intact; the remaining **mock** questions follow the same style to broaden coverage. All answers are in the *Answer Key* at the end — the question boxes are kept blank so the set can be used for self-testing.

Contents

Credit Assignment (real quiz)

From the credit-assignment / RUDDER quiz (DCA 2025), reproduced verbatim.

Question 1: Reward transformations preserving optimality

Which kind of transformation is *guaranteed* to leave the set of optimal policies unchanged?

- ☐ Clip to 1 all the rewards
- ☐ Any affine transformation of immediate rewards only
- ☐ A strictly monotonic transformation of the total return
- ☐ Squaring the reward at each step

Question 2: Potential-based reward shaping

The slides note that *potential-based reward shaping* is mathematically equivalent to...

- ☐ doubling the learning rate
- ☐ value-function normalization
- ☐ reward clipping
- ☐ initializing the Q-values with the potential function

Question 3: LSTM vs. FNN prediction errors

Compared with a feed-forward network (FNN), an LSTM used for contribution analysis tends to produce *lower-variance* differences in consecutive predictions. What statistical property of the LSTM's prediction errors explains this?

- ☐ Errors are *highly correlated*, so they cancel in the difference
- ☐ Errors have heavier tails, making large differences rare
- ☐ Errors are i.i.d. across time
- ☐ Errors are biased toward earlier time-steps

Question 4: Risk of online potential learning

The slides note that potential-based shaping is *equivalent to Q-value initialisation* $Q'(s,a) = Q(s,a) + \Phi(s)$. What practical risk does this equivalence highlight when Φ is *learned online* with TD-errors derived from sparse, delayed rewards?

- ☐ Bias is re-introduced because Φ itself must be bootstrapped from delayed feedback
- ☐ The learning rate must be halved to keep Q-updates unbiased
- ☐ Shaping may over-count rewards, double-updating the same transition
- ☐ Eligibility traces converge to zero faster, causing under-estimation

Question 5: The credit-assignment problem (Sutton & Minsky)

According to R. Sutton and Minsky, what is the problem of credit assignment? (Select one or more.)

- ☐ Assigning credit or blame for overall episodes that contributed to the expectation of the return
- ☐ Identifying models that get high rewards
- ☐ Assigning credit or blame for overall outcomes to each of the decisions that contributed to those outcomes
- ☐ Identifying actions that led to high rewards

Question 6: MC, TD, and reward delay

Which of the following sentences are true? (Select one or more.)

- ☐ Variance in MC increases with the delay of the reward
- ☐ TD methods correct the bias faster than MC
- ☐ TD(λ) assigns credit faster than TD(0)
- ☐ Variance in TD(0) increases with the delay of the reward

Question 7: Zero future reward and the Markov property

If we make the expected future rewards equal to zero (i.e. via reward redistribution), the Q-values are just the average of the immediate reward. Why is this *not* possible in a standard MDP setting? (Select one or more.)

- ☐ Because to have future expected reward zero, states must be enriched with the past, which breaks the Markov assumption
- ☐ It is completely possible in an MDP setting
- ☐ Because in this case the reward depends also on the previous state and action, and is therefore not Markov
- ☐ Because the reward is obtained before we enter the environment, and is therefore not causal, breaking the Markov property

Question 8: Definition of a sequence-Markov decision process (SDP)

What is the definition of a sequence-Markov decision process (SDP), as seen in class? (Select one or more.)

- ☐ A decision process equipped with a **Markov reward** and a **Markov policy**, but **transition probabilities** not required to be Markov
- ☐ A decision process equipped with a **Markov policy**, but **transition probabilities and reward** not required to be Markov
- ☐ A decision process equipped with a **Markov policy** and **Markov transition probabilities**, but a **reward** not required to be Markov
- ☐ A decision process equipped with a **Markov reward** and **Markov transition probabilities**, but a **policy** not required to be Markov

Question 9: Return-equivalent SDPs

What is the definition of **return-equivalent** SDPs? (Select one or more.)

- ☐ Two SDPs are return-equivalent if they differ only in their reward distributions but have the **same return per episode**
- ☐ Two SDPs are return-equivalent if they differ only in their reward distributions but have the **same expected return**
- ☐ Two SDPs are return-equivalent if they differ only in their reward distributions but have the **same expected return per episode**

Question 10: The explaining-away problem

What is the **explaining-away problem** in MDPs, when a function g is used to predict the return based on the sequence of actions and states? (Select one or more.)

- ☐ Since later states are used for return prediction, earlier actions causing reward are missed
- ☐ Previous actions can explain away future states, breaking the Markov property
- ☐ Since the sequence of states and actions are mutually independent, the prediction needs every component to explain the return
- ☐ In an MDP, one only needs the difference of states/actions to make a prediction, not the whole sequence

Question 11: LSTM vs. FFN for prediction

Which statements are true regarding differences between LSTM and feed-forward networks (FFN) for making future predictions? (Select one or more.)

- ☐ An LSTM will only learn to store information if a state-action pair has strong evidence for a change in the expected return
- ☐ FFNs are not suited for processing sequences
- ☐ LSTMs tend to have smaller prediction errors since they reuse past information
- ☐ FFNs are prone to prediction errors since they must extract information from every state to make a prediction at every timestep
- ☐ Prediction errors of LSTMs are more likely to cancel via prediction differences, since consecutive predictions rely on the same internal state and are highly correlated

Question 12: Sensitivity analysis vs. contribution analysis

What is the main difference between sensitivity analysis (SA) and contribution analysis (CA)? (Select one or more.)

- ☐ **SA** is gradient-based, while **CA** is not
- ☐ **CA** focuses on how a small change in the input changes the output, and **SA** focuses on how the input is responsible for the output
- ☐ **CA** is gradient-based, while **SA** is not
- ☐ **SA** focuses on how a small change in the input changes the output, and **CA** focuses on how the input is responsible for the output

Imitation Learning and Offline RL (mock)

Question 13: Compounding errors in behavioral cloning

A behavioral-cloning policy makes a mistake with probability ϵ under the expert's state distribution. According to Ross & Bagnell (2010), how does the total error grow with the horizon T?

- ☐ Linearly, $\mathcal{O}(T\epsilon)$, because errors are independent across steps
- ☐ Quadratically, $\mathcal{O}(T^2\epsilon)$, because an early mistake corrupts future states
- ☐ It stays bounded by ϵ regardless of T
- ☐ Logarithmically, $\mathcal{O}(\epsilon \log T)$

Question 14: What DAgger requires

DAgger reduces the $\mathcal{O}(T^2)$ error of behavioral cloning to $\mathcal{O}(T)$. What is the central cost or requirement that makes it applicable?

- ☐ A reward function for every state
- ☐ An expert that can be *queried for labels* on states the learner visits
- ☐ A known transition model of the environment
- ☐ A pretrained value function

Question 15: Multimodal demonstrations

An expert demonstrates passing an obstacle on either the left or the right for similar states. Why does a behavioral-cloning policy with an MSE loss fail here, and name one fix.

Question 16: Why standard off-policy RL fails offline

Running DQN or SAC on a fixed data set without new interaction typically diverges. What is the mechanism?

- ☐ The replay buffer is too small to cover the state space
- ☐ The $\max_{a'} Q(s', a')$ backup queries out-of-distribution actions, whose values the network overestimates, and the error propagates via bootstrapping
- ☐ The learning rate must decay faster offline
- ☐ Off-policy methods cannot use a discount factor offline

Question 17: Three families of offline RL

Name the three families of offline-RL methods and the single principle they all enforce.

Question 18: How IQL avoids OOD queries

Implicit Q-Learning (IQL) replaces the $\max_{a'} Q(s', a')$ target with expectile regression on a value function $V(s)$. Why does this avoid out-of-distribution action queries, and what does $V(s)$ approximate as the expectile $\tau \to 1$?

Question 19: Trajectory stitching

Which offline-RL approaches can perform *trajectory stitching* (combining sub-optimal trajectories into a better-than-dataset policy)?

- ☐ The Decision Transformer, because it conditions on return-to-go
- ☐ Bellman-backup methods (CQL, IQL), because value propagates through bootstrapping
- ☐ Only behavior cloning
- ☐ No offline method can stitch

Question 20: IL vs. offline RL

State the key difference in objective between imitation learning and offline RL, and which one can in principle exceed the quality of the data it is trained on.

Inverse Reinforcement Learning (real exam)

From the Inverse RL exam, reproduced verbatim with original answer choices.

Question 21: MaxEnt IRL with function approximation

When using Maximum Entropy IRL with high-dimensional function approximation (e.g. neural networks), one key bottleneck is:

- ☐ The inability to compute gradients with respect to ψ
- ☐ The need to differentiate through a complex partition function or to run repeated forward–backward algorithms
- ☐ The requirement that all expert trajectories be perfect
- ☐ The immediate convergence toward a uniform distribution over all trajectories

Question 22: Why Maximum Entropy IRL?

Why is Maximum Entropy IRL used?

- ☐ To ensure the reward function is constant across all states
- ☐ Because the method aims to retain as much randomness as possible, subject to matching expert statistics
- ☐ Because it cannot handle suboptimal demonstrations
- ☐ Because a Dirac delta distribution over the best path is always infeasible

Question 23: A common challenge in IRL

What is a common challenge when performing IRL?

- ☐ There is always a single unique reward function consistent with the expert's behavior
- ☐ The learned reward is trivial to evaluate without further policy optimization
- ☐ Expert demonstrations may be suboptimal, complicating reward inference
- ☐ IRL does not require knowledge or assumptions about the environment's dynamics

Question 24: Unique aspect of IRL vs. imitation learning

Compared to Imitation Learning (IL), a unique aspect of IRL is that IRL:

- ☐ Aims to learn the expert's reward rather than just copying actions
- ☐ Only works in deterministic environments
- ☐ Discards the idea of a reward function entirely
- ☐ Ignores expert demonstrations

Question 25: Role of $p(\tau)$ in MaxEnt IRL

In Maximum Entropy IRL, the probability of a trajectory τ is written as $p_\psi(\tau \mid \mathcal{O}_{0:T}) \propto p(\tau)\exp(r_\psi(\tau))$. What is the role of $p(\tau)$ in this expression?

- ☐ It enforces that all trajectories with the same reward get zero probability
- ☐ It encodes the environment's dynamics or "how likely" the raw trajectory is, independent of reward
- ☐ It ensures that the partition function is always 1
- ☐ It guarantees that expert demonstrations appear with higher probability than random trajectories by default

Question 26: Slack variable in maximum-margin IRL

Which statement correctly describes a slack variable ζ in maximum-margin IRL?

- ☐ It controls how strictly the constraints must be satisfied, allowing the expert to be suboptimal
- ☐ It ensures the maximum margin has infinite size
- ☐ It forces the expert's trajectories to always have higher reward
- ☐ It is irrelevant in IRL and only used in supervised SVMs

Question 27: What IRL starts with

In IRL, we typically start with:

- ☐ A set of expert demonstrations (trajectories)
- ☐ A fully known reward function from the environment
- ☐ A labeled dataset with correct actions for each state
- ☐ No knowledge about the environment's dynamics

Question 28: Interpreting the MaxEnt IRL gradient

Consider the gradient of the Maximum Entropy IRL objective, $\mathbb{E}_{\tau\sim\pi^*}[\nabla_\psi r_\psi(\tau)] - \mathbb{E}_{\tau\sim p^\psi(\tau|\mathcal{O}_{0:T})}[\nabla_\psi r_\psi(\tau)]$. Conceptually, what does each term represent?

- ☐ The first term decreases the expert's reward, the second increases the current policy's reward
- ☐ Both terms push the reward parameters in the same direction
- ☐ The first term reinforces expert trajectories; the second penalizes the policy's own trajectories
- ☐ They both approximate the state visitation frequencies

Question 29: What makes MaxEnt IRL log-likelihood hard

When optimizing the log-likelihood in Maximum Entropy IRL, what makes this problem computationally challenging?

- ☐ The partition function Z involves a sum over all possible trajectories, which can be extremely large
- ☐ The gradient of the likelihood does not depend on ψ

- ☐ Each demonstration is guaranteed to be optimal
- ☐ The objective always converges in one iteration

Question 30: Feature matching with a linear reward

In feature-matching IRL with a linear reward $r_\psi(s,a) = \psi^\mathsf{T} f(s,a)$, which statement is true?

- ☐ The reward is assumed to be a non-linear function
- ☐ We aim to match the expert's feature expectations $\mu(\pi)$
- ☐ We never need to solve an MDP to update ψ
- ☐ The reward depends only on the next state transition

Question 31: Purpose of the structured margin term

The structured max-margin IRL formulation uses a loss term $D(\pi_{\text{expert}}, \pi)$ in $\psi^\mathsf{T}\mu(\pi_{\text{expert}}) > \max_\pi(\psi^\mathsf{T}\mu(\pi) + D(\pi_{\text{expert}}, \pi))$. What is the purpose of $D(\pi_{\text{expert}}, \pi)$?

- ☐ It ensures that only the final reward function matters
- ☐ It differentiates between policies that are "almost" like the expert versus those that are very different
- ☐ It nullifies any margin gained by suboptimal policies
- ☐ It speeds up the solver by removing constraints

Question 32: Primary objective of max-margin IRL

In the maximum-margin approach to IRL (inspired by SVMs), what is the primary objective?

- ☐ To find the widest margin in the raw observations of states
- ☐ To maximize the margin between the expert's feature expectations and those of any other policy
- ☐ To minimize the number of slack variables to zero
- ☐ To guarantee that the expert policy is identical to all alternative policies

Question 33: Primary goal of IRL

Which of the following is a primary goal of Inverse Reinforcement Learning?

- ☐ Recover the reward function from expert demonstrations
- ☐ Perform clustering on reward signals
- ☐ Directly mimic the expert's actions without any reward model
- ☐ Randomize the learned policy to increase exploration

Preference-Based RL, RLHF and DPO (mock)

Question 34: The Bradley–Terry preference model

A reward model is trained from pairwise preferences $\tau_w \succ \tau_l$. Write the Bradley–Terry probability that τ_w is preferred in terms of the trajectory rewards $r_\theta(\tau_w), r_\theta(\tau_l)$, and state what loss is used to fit it.

Question 35: The KL penalty in RLHF

The RLHF objective is $\max_\theta \mathbb{E}_{y\sim\pi_\theta}[\, r_\phi(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\,]$. What is the role of the KL term, and what failure mode does it mitigate?

- ☐ It increases the entropy of the reward model
- ☐ It keeps the policy close to the reference (SFT) model, mitigating reward overoptimization
- ☐ It guarantees the reward model is unbiased
- ☐ It replaces the need for a value function in PPO

Question 36: Two anchors in PPO-style RLHF

PPO-style RLHF uses two distinct reference points. Identify each and what it controls.

Question 37: What DPO eliminates

Direct Preference Optimization (DPO) rewrites the RLHF problem so that the reward and partition function cancel. What does this eliminate compared to standard RLHF, and what is the main limitation of DPO?

- ☐ It eliminates the SFT step; limitation: it needs more human labels
- ☐ It eliminates the separate reward model and the RL loop, becoming supervised learning on preference pairs; limitation: it assumes a fixed offline preference set
- ☐ It eliminates the KL penalty; limitation: it diverges
- ☐ It eliminates pretraining; limitation: it cannot scale

Question 38: Goodhart's law and reward hacking

Give the statement of Goodhart's law as it applies to RL, and one concrete example of reward hacking discussed in the lecture.

Question 39: GRPO vs. PPO

Group Relative Policy Optimization (GRPO) avoids one expensive component of PPO-style RLHF. Which component, and how does GRPO obtain its advantage estimate instead?

- ☐ It avoids the reward model; it uses human labels directly
- ☐ It avoids the separate value (critic) network; it standardizes rewards within a group of sampled outputs for the same prompt
- ☐ It avoids the KL penalty; it clips more aggressively
- ☐ It avoids sampling; it uses a fixed dataset

Question 40: RLAIF and Constitutional AI

What problem does RLAIF address, and what is the key insight that makes it work?

Planning and Search (mock)

Question 41: The four phases of MCTS

List the four phases of Monte Carlo Tree Search in order, and state which action is finally played at the root.

Question 42: The UCT tree policy

UCT selects actions by $a^* = \arg\max_a [\, Q(s,a) + c\sqrt{\ln N(s)/N(s,a)}\,]$. Explain what each of the two terms encourages and the role of c.

Question 43: AlphaZero's network

How does AlphaZero differ from AlphaGo, and what does its single network $f_\theta(s)$ output?

- ☐ AlphaZero adds a human-games dataset; the network outputs only a value
- ☐ AlphaZero removes human games, the rollout policy, and supervised pretraining; one network outputs both a policy prior $\mathbf{p}$ and a value v
- ☐ AlphaZero replaces MCTS with Q-learning; the network outputs Q-values
- ☐ AlphaZero uses two separate networks trained on expert data

Question 44: MCTS as policy improvement

In AlphaZero, why is MCTS described as a "policy-improvement operator," and what role does the network play with respect to the search?

Question 45: The branching problem

At depth d with branching factor b, a full search tree has how many nodes, and name the two classical remedies MCTS uses to make the search tractable. _____

Question 46: Random shooting vs. CEM

For continuous-control trajectory optimisation, contrast random shooting with the Cross-Entropy Method (CEM). What does CEM do that random shooting does not?

Question 47: Model Predictive Control

In MPC, the agent optimizes an H-step action sequence but executes only part of it. Which part, and why does this help when the model is imperfect?

- ☐ It executes the whole sequence open-loop; this is faster
- ☐ It executes only the first action, then re-plans, which corrects for model errors after each real transition
- ☐ It executes the last action; this maximizes lookahead
- ☐ It executes a random action from the sequence for exploration

Model-Based RL (mock)

Question 48: Dyna-Q and the planning steps

In Dyna-Q, real experience updates the model and the Q-function, while imagined experience updates only the Q-function. What does the algorithm reduce to when the number of planning steps $K = 0$?

- ☐ Monte Carlo control
- ☐ Ordinary model-free Q-learning
- ☐ Policy iteration
- ☐ Behavioral cloning

Question 49: Why MBPO uses short rollouts

MBPO starts model rollouts from real replay-buffer states and keeps them very short (often $k = 1$–5). Explain the trade-off that motivates short rollouts.

Question 50: Aleatoric vs. epistemic uncertainty

Define aleatoric and epistemic uncertainty, and state which one can be reduced by collecting more data and which one matters most for model-based RL planning.

Question 51: Ensembles and uncertainty

A single Gaussian neural-network model outputs a mean and variance. Which type of uncertainty does its predictive variance capture, and what must you do instead to estimate epistemic uncertainty?

- ☐ It captures epistemic uncertainty; nothing more is needed
- ☐ It captures aleatoric uncertainty; for epistemic uncertainty you need disagreement between an ensemble of models
- ☐ It captures both equally; ensembles are unnecessary
- ☐ It captures neither; only Bayesian neural networks work

Question 52: Model exploitation

What is model exploitation in model-based RL, and name two mitigations.

Question 53: PETS

PETS combines three ingredients for continuous control. Name them and state what "trajectory sampling" (rolling out particles through the ensemble) achieves.

Question 54: Latent world models

Why do agents like Dreamer and MuZero learn a *latent* state z_t instead of predicting in the raw observation space? Give two reasons.

RL for Foundation Models (mock)

Question 55: Autoregressive generation as RL

Map an autoregressive language model onto the RL formalism: what plays the role of the policy, the state, the action, and the reward?

Question 56: Pretraining as imitation

Why is language-model pretraining described as a form of behavioral cloning, and why is it "not enough" to produce a helpful assistant?

Question 57: GRPO advantage normalization

In GRPO, G outputs are sampled for the same prompt x and the advantage is $\hat{A}_i = (r_i - \text{mean}(r_{1:G}))/(\text{std}(r_{1:G}) + \delta)$. At what level (token or sequence) is $\hat{A}_i$ computed, and how is it applied to the tokens of output y_i?

- ☐ Token-level; applied independently to each token
- ☐ Sequence-level; the same $\hat{A}_i$ is applied to all tokens of output y_i

☐ It is not an advantage at all; it is a reward

☐ It is computed across prompts, not within a group

Question 58: Verifiable rewards (RLVR)

What is the key idea of Reinforcement Learning with Verifiable Rewards (RLVR), and why is a composite reward $r(y) = \lambda_{\text{format}} r_{\text{format}}(y) + \lambda_{\text{correct}} r_{\text{correct}}(y)$ often used?

Question 59: SFT vs. RLHF vs. RLVR

For each of SFT, RLHF, and RLVR, give the training signal it relies on and its main risk.

Question 60: Generalist agents

Gato uses one Transformer with the same weights across Atari, robotics, captioning, and dialogue. What makes this possible, and how was Gato primarily trained?

Answer Key

Real-exam answers (Q1–12 credit assignment, Q21–33 IRL) follow the official keys; mock answers give the intended solution.

Credit Assignment (real)

Q1. **C** — a strictly monotonic transformation of the *total return* preserves the argmax over policies; affine/clipping/squaring of per-step rewards need not.

Q2. **D** — potential-based shaping with Φ is equivalent to initializing Q-values with the potential function.

Q3. **A** — the LSTM's consecutive prediction errors are highly correlated, so they cancel in the difference, giving lower variance.

Q4. **A** — if Φ is learned online from sparse, delayed TD-errors, it must itself be bootstrapped, which re-introduces bias.

Q5. **C** — credit assignment is assigning credit/blame for overall outcomes to each of the decisions that contributed.

Q6. **A, C** — variance in MC grows with reward delay; TD(λ) assigns credit faster than TD(0).

Q7. **C** — with zero expected future reward the reward depends on the previous state/action, so it is no longer Markov.

Q8. **C** — an SDP has a Markov policy and Markov transitions, but the reward need not be Markov.

Q9. **B** — return-equivalent SDPs differ only in reward distribution but have the same *expected return.*

Q10. **A** — because later states are used for prediction, earlier reward-causing actions get explained away (missed).

Q11. **A, B, C, D, E** — all statements about LSTM vs. FFN prediction are correct.

Q12. **D** — SA measures how a small input change affects the output; CA measures how the input is *responsible* for the output.

Imitation Learning and Offline RL (mock)

Q13. **B** — error grows quadratically, $\mathcal{O}(T^2\epsilon)$, because mistakes change future states.

Q14. **B** — DAgger needs an expert that can label the learner's on-policy states.

Q15. MSE *averages* the two valid modes ($\approx \frac{1}{2}(a_{\text{left}} + a_{\text{right}})$), driving into the obstacle. Fix: mixture-of-Gaussians head, latent-variable model (CVAE/flow), or autoregressive action discretization.

Q16. **B** — the $\max_{a'} Q$ backup queries OOD actions; the net overestimates them and the error propagates through bootstrapping.

Q17. Behavior regularization (stay close to π_β), conservative value estimation (CQL), and sequence modeling (Decision Transformer). Common principle: *do not trust the model at out-of-distribution inputs.*

Q18. IQL regresses $V(s)$ using *expectile* loss over *dataset actions only*, so no unseen action is ever evaluated; as $\tau \to 1$, $V(s)$ approaches a soft $\max_{a \in \mathcal{D}} Q(s, a)$.

Q19. **B** — Bellman-backup methods (CQL, IQL) can stitch; the Decision Transformer cannot.

Q20. IL matches the expert (upper bound: expert level); offline RL maximizes reward and can in principle exceed the dataset's behavior policy.

Inverse Reinforcement Learning (real)

Q21. **B** — differentiating through the partition function / repeated forward–backward passes.

Q22. **B** — retain maximum randomness subject to matching expert statistics.

Q23. **C** — expert demonstrations may be suboptimal.

Q24. **A** — IRL learns the expert's reward rather than copying actions.

Q25. **B** — $p(\tau)$ encodes environment dynamics / how likely the raw trajectory is, independent of reward.

Q26. **A** — the slack variable relaxes the constraints, allowing a suboptimal expert.

Q27. **A** — IRL starts with a set of expert demonstrations (trajectories).

Q28. **C** — first term reinforces expert trajectories; second penalizes the policy's own trajectories.

Q29. **A** — the partition function Z sums over all possible trajectories.

Q30. **B** — we aim to match the expert's feature expectations $\mu(\pi)$.

Q31. **B** — D differentiates "almost-expert" policies from very different ones (structured margin).

Q32. **B** — maximize the margin between the expert's feature expectations and those of any other policy.

Q33. **A** — recover the reward function from expert demonstrations.

Preference-Based RL, RLHF and DPO (mock)

Q34. $P(\tau_w \succ \tau_l) = \sigma(r_\theta(\tau_w) - r_\theta(\tau_l))$, with $r_\theta(\tau) = \sum_{(s,a)\in\tau} r_\theta(s,a)$; fit by maximizing log-likelihood (binary cross-entropy on the preference label).

Q35. **B** — the KL term keeps π_θ near the SFT reference, mitigating reward overoptimization.

Q36. KL penalty: π_θ vs. π_{ref} (SFT) — stay close to the trusted assistant. PPO clipping: π_θ vs. $\pi_{\theta_{\text{old}}}$ — keep each update step stable.

Q37. **B** — DPO removes the separate reward model and the RL loop (pure supervised learning on pairs); limitation: it assumes a fixed offline preference set.

Q38. "When a measure becomes a target, it ceases to be a good measure." Example: the OpenAI boat-race agent spinning to collect speed boosts instead of finishing (or a verbose/sycophantic RLHF model; a robot moving the camera so an object only *appears* grasped).

Q39. **B** — GRPO drops the value/critic network and standardizes rewards within a group of sampled outputs for the same prompt.

Q40. RLAIF replaces human annotators with a strong LLM because human labelling does not scale; key insight: critique is easier than generation. Constitutional AI encodes the values as written principles the model uses to critique and revise its own outputs.

Planning and Search (mock)

Q41. Selection (tree policy / UCT) → Expansion → Simulation (rollout) → Backpropagation. The action with the *highest visit count* at the root is played.

Q42. $Q(s,a)$ exploits promising branches; $c\sqrt{\ln N(s)/N(s,a)}$ explores under-visited actions (large when $N(s,a)$ is small). c sets the exploration–exploitation balance.

Q43. **B** — AlphaZero removes human games, the rollout policy, and supervised pretraining; one network outputs both a policy prior $\mathbf{p}$ and a value v.

Q44. MCTS turns the raw network policy $\mathbf{p}_\theta$ into a stronger search policy $\pi_{\text{MCTS}} \propto N(s,a)^{1/\tau}$; the network is then trained to *amortize* (imitate) that improved search and the final outcome.

Q45. b^d nodes. Remedies: depth reduction (a learned value function replaces deep rollouts) and breadth reduction (expand only promising branches).

Q46. Random shooting samples action sequences uniformly and keeps the best; CEM *iteratively refits* the sampling distribution to the elite (best-scoring) sequences, concentrating samples on good regions.

Q47. **B** — MPC executes only the first action, then re-plans, correcting model errors after each real transition.

Model-Based RL (mock)

Q48. **B** — with $K = 0$ Dyna-Q is ordinary model-free Q-learning.

Q49. Short rollouts trade off model usefulness against compounding error: longer rollouts give more imagined data but accumulate model error and invite exploitation. Starting from real states and imagining only a few steps keeps synthetic data trustworthy. Goal: use the model only where it is accurate.

Q50. Aleatoric = inherent environment randomness (not reducible with more data); epistemic = uncertainty from limited data / OOD queries (reducible). Epistemic uncertainty matters most for MBRL — it tells the planner where the model should not be trusted.

Q51. **B** — a single Gaussian net captures aleatoric uncertainty (predictive variance); epistemic uncertainty requires disagreement across an ensemble.

Q52. Model exploitation: a policy optimized against a learned model finds and exploits its errors (e.g. a glitch giving infinite simulated reward). Mitigations: limit the planning horizon, penalize uncertainty (pessimistic planning), and frequently retrain on new real data.

Q53. Probabilistic ensemble (dynamics + uncertainty), CEM (action-sequence search), and trajectory sampling (ensemble rollouts). Trajectory sampling propagates model disagreement into the planning, so uncertain actions are down-weighted.

Q54. Raw observations are high-dimensional, partial, contain irrelevant variation, and may miss history-dependent information; a latent z_t summarizes the useful information for control (the goal is a control-useful representation, not perfect reconstruction).

RL for Foundation Models (mock)

Q54. Policy = $\pi_\theta(y_t \mid x, y_{<t})$ (the LM); state = prompt + generated prefix; action = next token (or tool call); reward = preference / verifier / task-success signal.

Q55. Pretraining maximizes token likelihood on text, i.e. it *imitates* the conditional distribution of the data — behavioral cloning. It is not enough because it learns plausible (and conflicting) continuations, not instruction-following, calibration, refusal, or alignment with preferences.

Q56. **B** — $\hat{A}_i$ is sequence-level (one value per output) and the same $\hat{A}_i$ is applied to all tokens of y_i.

Q57. RLVR replaces a learned preference reward with an automatic checker (math verifier, unit tests, API check, game score). The composite reward teaches both *format* (answer must be extractable/parseable) and *correctness* (extracted answer passes the verifier); without a format term, capability is underestimated.

Q58. SFT: demonstrations — risk: limited by examples. RLHF: human preferences — risk: proxy reward hacking / overoptimization. RLVR: automatic verifier — risk: narrow rewards and hacking the verifier.

Q59. Tokenizing all modalities (text, vision, state, actions) into a shared sequence makes language modeling and action prediction structurally identical, so one Transformer can serve all tasks. Gato was trained primarily by supervised sequence modeling (imitation) on mixed datasets.